Classification of Leopard Frogs Native to Colorado: An Exploratory Analysis

Matthew Blong & Connor Greenhalgh

Table of Contents

Abstract 2
Introduction 2
Summary Statistics 3
Statistical Methods 3
Limitations and Alternatives 5
Results and Conclusions 6
Authors' Statements 9
References 10
Appendix 11

Abstract

Declining population rates in amphibians have led researchers and conservationists to increase their efforts in addressing this issue on a species-specific level. In the case of Colorado Leopard Frogs, ecological sympatry due to environmental factors such as climate change has resulted in hybridization between Northern and Plains Leopard Frogs, which makes species classification a formidable challenge. Researchers have set out to find cost-effective ways to classify species through capturing frogs in the field and documenting simple morphological traits. In this exploratory analysis, we investigated three machine learning methods (Random Forest, Multinomial Logistic Regression, and GLMET) to optimize classification between Northern Leopard Frogs, Plains Leopard Frogs, and hybrids. High accuracy metrics, such as precision, recall, F1 score, and balanced accuracy, across species and inherent feature selection leads us to believe that GLMNET is the best machine learning method for leopard frog species classification using categorical physical identifiers. As a result, we illustrate that a spot on a leopard frog's tympanum and dorsal frontal brokenness are traits that best predict species identification and that, due to virtually perfect classification accuracy, Northern Leopard Frogs are distinguishable by morphology alone.

Introduction

Amphibians are declining on a global scale, where a variety of ecological factors threaten roughly 40% of amphibian species (Ward et al. 2026). Locally, Colorado leopard frogs have been a subject of particular concern. There are four main reasons for this species' endangerment. First, rising population growth in Front Range metropolises encroaches on wetland habitats native to leopard frogs. Second, increasing amounts of Chytrid bacteria interfere with leopard frogs' respiration through keratinizing its skin, functionally causing cardiac arrest. Third, species such as bull frogs outcompete leopard frogs for resources as well as predate on leopard frogs as an additional food source. Lastly, drought due to climate change in the plain region threatens living conditions.

There are two main species of leopard frogs native to Colorado: Northern Leopard Frogs (NLF) and Plains Leopard Frogs (PLF). Both species have natural sympatry in southeast Colorado. However, with the exacerbating effects of climate change, this ecological overlap is becoming more pronounced. A natural consequence of this sympatry is interbreeding between frog species, leading to hybrid leopard frogs (HYB). Hybridization between these two species concerns conservationists due to high-risk consequences such as genetic swamping, decreased population growth rates, loss of adaptive traits, and even local extinction. As hybridization increases, conservationists find it more difficult to differentiate between these two species, which makes species-specific population threats harder to diagnose and address.

The following observational study involved capturing and documenting 50 randomly selected leopard frogs from various leopard frog habitats across Colorado and categorizing species based on factor-level traits (Table 2). The species in our dataset are imbalanced with 20 NLF, 20 PLF, and 10 HYB. The primary goal of our analysis is to identify which combination of morphological traits best predicts leopard frog species: plains, northern, or hybrid leopard frog. By comparing multiple machine learning techniques, we will determine which model classifies frog species the most accurately, which subsequently through feature selection, will determine the most contributive morphological features to species classification. In addition, we want to determine which species, if any, can be identified by morphology alone.

Summary Statistics

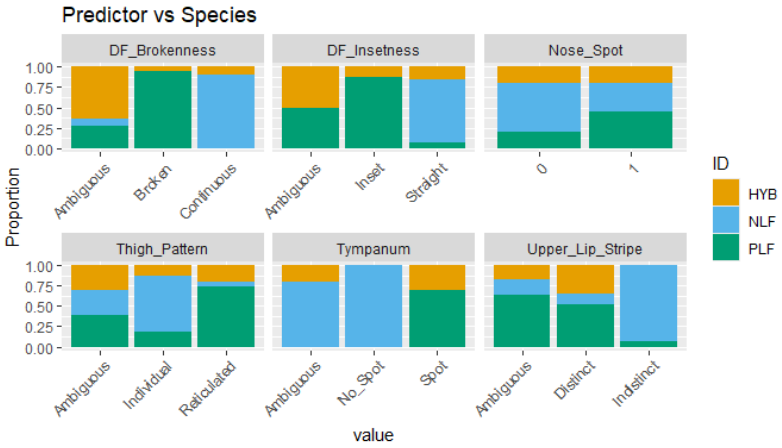


Figure 1: Relative frequency of variables and factor-level by species

Table 1: Counts and proportions of each variable and factor level by species

	HYB (N=10)	NLF (N=20)	PLF (N=20)	Overall (N=50)
Nose Spot				
0	2 (20.0%)	6 (30.0%)	2 (10.0%)	10 (20.0%)
1	8 (80.0%)	14 (70.0%)	18 (90.0%)	40 (80.0%)
Thigh Pattern				
Ambiguous	4 (40.0%)	4 (20.0%)	5 (25.0%)	13 (26.0%)
Individually Spotted	3 (30.0%)	15 (75.0%)	4 (20.0%)	22 (44.0%)
Reticulated	3 (30.0%)	1 (5.0%)	11 (55.0%)	15 (30.0%)
Upper Lip Stripe				
Ambiguous	2 (20.0%)	2 (10.0%)	7 (35.0%)	11 (22.0%)
Distinct in front of eye	8 (80.0%)	3 (15.0%)	12 (60.0%)	23 (46.0%)
Indistinct in front of eye	0 (0%)	15 (75.0%)	1 (5.0%)	16 (32.0%)
Tympanum				
Ambiguous	1 (10.0%)	4 (20.0%)	0 (0%)	5 (10.0%)
No Spot	0 (0%)	16 (80.0%)	0 (0%)	16 (32.0%)
Spot	9 (90.0%)	0 (0%)	20 (100%)	29 (58.0%)
DF Insetness				
Ambiguous	4 (40.0%)	0 (0%)	4 (20.0%)	8 (16.0%)
Inset	2 (20.0%)	0 (0%)	14 (70.0%)	16 (32.0%)
Straight	4 (40.0%)	20 (100%)	2 (10.0%)	26 (52.0%)
DF Brokenness				
Ambiguous	7 (70.0%)	1 (5.0%)	3 (15.0%)	11 (22.0%)
Broken	1 (10.0%)	0 (0%)	17 (85.0%)	18 (36.0%)
Continuous	2 (20.0%)	19 (95.0%)	0 (0%)	21 (42.0%)

Statistical Methods

Our original dataset included 12 variables. However, to consolidate information, we combined variables that were divided into right and left sides. This applied to 'DF Insetness,' 'DF Brokenness,' and 'Tympanum,' where if both right and left sides matched, then the new combined variable would be the matching factor, and if right and left sides were different, the new combined variable would be labeled as 'Ambiguous.' Researchers recommended this method to reduce the number of variables with minimal loss of information (Ward et al. 2026).

Table 2: Description of each variable and use in analysis

Variable Name	Description	Variable Class	Use in Analysis

ID	The species of each frog	Factor (3 levels: NLF, PLF, HYB)	Response
SVL	Snout to vent length of each frog	Numeric	Not Applicable
DF Insetness	Categorizes the Dorsal Front Insetness	Factor (3 levels: Inset, Straight, Ambiguous)	Predictor
DF Brokenness	Categorizes the Dorsal Front Brokenness	Factor (3 levels: Broken, Continuous, Ambiguous)	Predictor
Nose Spot	Indicates whether the frog has a spot on its nose	Factor (2 levels: 1, 0)	Predictor
Thigh Pattern	Describes the thigh pattern on each frog	Factor (3 levels: Reticulated, Individually Spotted, Ambiguous)	Predictor
Tympanum	Describes whether the tympanum of each frog has a spot	Factor (3 levels: Spot, No Spot, Ambiguous)	Predictor
Upper Lip Stripe	Categorizes the distinctness of a stripe on each frog's upper lip	Factor (3 levels: Distinct in front of eye, Indistinct in front of eye, Ambiguous)	Predictor

All relevant predictors are factor variables. For each variable, researchers captured, collected, and classified 50 Colorado leopard frogs according to the factor levels listed above (Table 2). Researchers, then, determined leopard frog species based on location of collection and combination of expected traits. Snout to vent length (SVL) is not relevant to our analysis because researchers only collected this variable to determine whether the frogs were adults as non-adult frogs were discarded and not used in their data collection process (Ward et al. 2026). 'Individual', a unique identifier for each frog, was not used in the classification process. After variable reduction, we used 6 predictor variables for classifying our response (ID). We predicted the species for each frog using several machine learning methods such as Random Forest, multinomial logistic regression, and GLMNET. All models have minimal parametric assumptions: researcher's collection process ensured independent observations and the combination of right and left variables eliminated concern for multicollinearity.

Due to small sample size (n=50), we prioritized k-fold cross validation over a train-test split to decrease variance in our estimates and increase reliability in model performance. We used the *trainControl* function from the *caret* package in R to define resampling and training parameters (Kuhn, 2008). With this function, we conducted cross validation with repeated training/test splits (5 k-folds with 10 repetitions with the *repeatedcv* method) for multiclass classification since overfitting is a concern with only 50 species recorded. In addition, we confirmed researchers' above-mentioned intuition to combine variables since feature reduction can help prevent overfitting by reducing complexity and preventing memorization of noise (GeeksforGeeks, 2024). To optimize predication accuracy, we trained our models over a tuning grid over a set of possible hyperparameter values. For each machine learning method, we extracted the best tuned model by optimizing mean balanced accuracy (explained below), rank variable importance (computed by *varImp* from the caret package), confusion matrices, and our model performance metrics.

The primary metric for comparison is prediction accuracy. Though accuracy is a key metric in measuring model performance, it is often incomplete in its assessment of model dependability, especially with imbalanced data (Babu 2025). As a result, we evaluated the following models using other measures including precision, recall, F1 Score, and balanced accuracy to provide a more holistic comparison between models. Precision is "the proportion of all the model's positive classifications that are actually positive" (Google, 2024). Recall is the true positive rate, which is correctly labeled classes over both correctly and incorrectly labeled classes. F1 Score is "the

harmonic mean…of precision and recall." Balanced accuracy averages recall and specificity, the true negative rate (Bobbitt, 2021). In addition, since we want to determine if any of the three species can be classified through morphology alone, we evaluate model performance for each class as well. The above calculations were computed using the results from our confusion matrices for each model (as shown in Figure 2).

Our first classification model is the machine learning method Random Forest. Since Random Forest handles categorical variables well, we did not encode our predictors into dummy variables. Random Forest combines the results of multiple decision trees, each analyzing different parts of the data. For classification, predictions are based on the category for which the majority of trees vote (GeeksforGeeks, 2024). Then, we train a Random Forest model with the ranger method, which is "a fast implementation of random forests" (ranger function, n.d.). Our tuning hyperparameters for Random Forest include *mtry*, *splitrule*, and *min.node.size*. *mtry* is the "Number of Randomly Selected Predictors" to split a tree at any particular node (Kuhn, 2016). *splitrule* is the algorithm that decides where the features in each decision tree splits to best optimize classification. *min.node.size* is the "[v]alue of minimal node size used" (ranger, n.d.), which we kept fixed at the default value of one. The contribution of each variable in a Random Forest model is calculated by ranking mean decrease in Gini (Prabhakaran, 2026), which is "defined as the total decrease in node impurity averaged over all trees of the ensemble" (Cho & Su, n.d.). As with all variable importance metrics for the following models, importance scores are scaled from zero to 100, where variables closer to 100 are more important for classification (Kuhn, 2019). As a side note, since our predictors are categorical, *caret* produces multiple importance scores for each factor level (Prabhakaran, 2026).

Our next model uses multinomial logistic regression, which predicts class probabilities through "a link function to mathematically connect the linear combination of input variables and their coefficients (known as the linear predictor) to the expected probability of each category" (Frost, 2024). To ensure proper model fitting, multinomial logistic regression requires us to encode our categorical predictors through a method such as one-hot encoding. We use the same train control as mentioned above to set training parameters as well as perform cross validation. Our only tuning hyperparameter is *decay*, which does not limit the number of parameters the model uses to classify data points but rather "operates by restricting the values that the parameters can take" (Aston et al., 2023). Variable importance for multinomial logistic regression is calculated using "combinations of the absolute values of the weights" (Kuhn, 2016).

The final model in our analysis is GLMNET, which fits multinomial logistic regression with maximum likelihood penalties: the "ridge penalty shrinks the coefficients of correlated predictors towards each other while the lasso tends to pick one of them and discard the others" (Hastie et al., 2026). This model has two hyperparameters to train: *alpha* and *lambda*. *alpha* controls the ratio between ridge and lasso penalties, where 0 is a complete ridge penalty and 1 is a complete lasso and anything in between represents the mixture between the two penalties. *lambda* controls the strength of this penalty parameter and can take on values zero or larger (Campbell, 2025). Variable importance is calculated by ranking the "absolute value of the coefficients corresponding [sic.] the tuned model" (Kuhn, 2016).

Limitations and Alternatives

In terms of data collection, there are fewer observations for machine learning models to train due to a small sample size (n=50). Thus, overfitting is a concern although our moderate accuracy

metrics make us less suspicious since we took measures to address this issue such as k-fold repeated cross validation and variable reduction. However, a larger sample size would have allowed us to consider hold-out data. In addition, we suspect that had our sample size been larger, our prediction accuracy would have been higher since our machine learning algorithms, again, would have been better able to capture the variability within our data and capture the true distribution of leopard frogs in Colorado. In regard to model selection, machine learning algorithms are complex and can lack accessible interpretability, especially with those unfamiliar with these methods such as ecological researchers. Perhaps, a method computing a single decision tree would be an improvement on this front although one may forsake accuracy as a result. However, as we show below, we can infer a flowchart-type logic similar to a single decision tree through our analysis of variable importance, especially with GLMNET.

Results and Conclusions

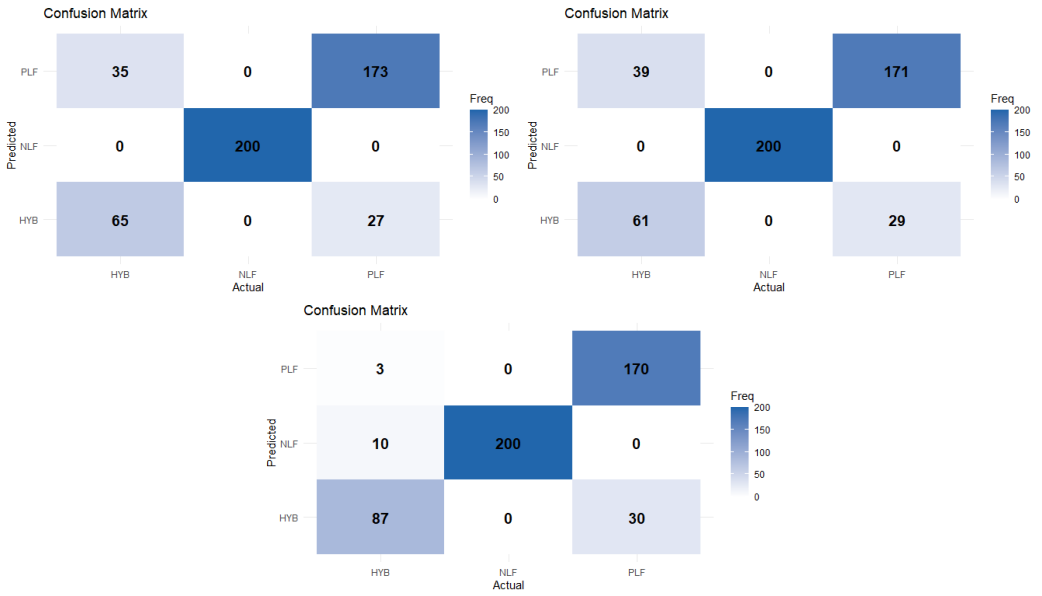


Figure 2: Confusion matrices for each model: Random Forest (top left), Multinomial Logistic Regression (top right), and GLMNET (bottom)

Table 3: Model Performance Comparison

Species	Metric	Random Forest	MLR	GLMNET
HYB	Balanced Accuracy	0.791	0.769	0.897
	F1	0.677	0.642	0.802
	Precision	0.707	0.678	0.744
	Sensitivity	0.650	0.610	0.870
	Specificity	0.932	0.927	0.925

Species	Metric	Random Forest	MLR	GLMNET
NLF	Balanced Accuracy	1.000	1.000	0.983
	F1	1.000	1.000	0.976
	Precision	1.000	1.000	0.952
	Sensitivity	1.000	1.000	1.000
	Specificity	1.000	1.000	0.967
PLF	Balanced Accuracy	0.874	0.863	0.920
	F1	0.848	0.834	0.912
	Precision	0.832	0.814	0.983
	Sensitivity	0.865	0.855	0.850
	Specificity	0.883	0.870	0.990
Overall	Balanced Accuracy	0.888	0.877	0.934
	F1	0.842	0.825	0.896
	Precision	0.846	0.831	0.893
	Sensitivity	0.838	0.822	0.907
	Specificity	0.939	0.932	0.961

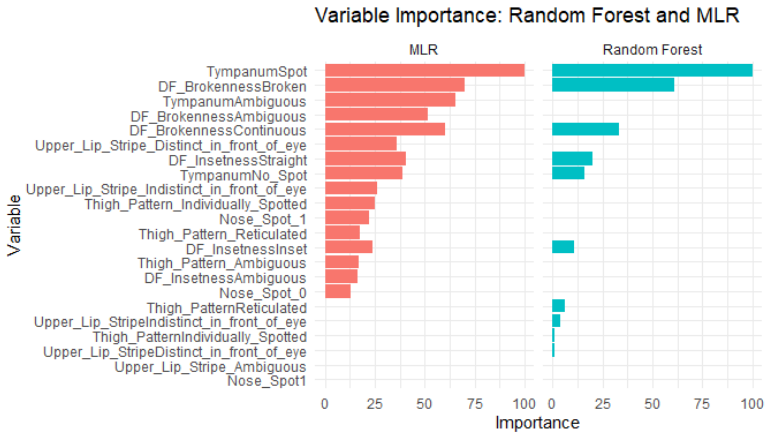


Figure 3: Variable importance for Random Forest and multinomial logistic regression models

The best tuned model for Random Forest with an overall accuracy of 0.872 (0.8395, 0.9) had *mtry* = 4, *splitrule* = 'extratrees', and *min.node.size* = 1. Four predictors were randomly selected to split decision trees. The split rule is 'extratrees,' which "trains numerous decision trees and aggregates the results from the group of decision trees to output a prediction" (Thankachan, 2022). The value of minimal node size is one. Figure 3 illustrates that our most important variables that contribute toward the accuracy of our predictions for this model are *Tympanum – Spot* (100), DF Brokenness–Broken (68.87), and *DF Brokenness – Continuous* (47.74). As stated above, this means that *Tympanum – Spot* had the highest mean decrease in Gini. In other words, the presence of a spot on the frog's tympanum was crucial to splitting the model into pure nodes, highly contributing to better predictions. In addition, both when a frog's dorsal front was broken and continuous substantially contributed toward decreasing mean Gini. However, it is worth noting that Figure 3 also shows that other variables contributed toward node purity although to a smaller extent than the above-stated variables.

The best tuned model for Multinomial Logistic Regression with an overall accuracy of 0.864 (0.8308, 0.8928) had *decay* = 0.1. Our best tuned model exhibited stable performance with a larger amount of regularization, which suggests that simpler models are sufficient for the data (Kuhn & Johnson, 2016). Our most important variables for this model are *Tympanum – Spot* (100), *DF Brokenness – Broken* (70.32), *Tympanum – Ambiguous* (65.66), and *DF Brokenness – Continuous* (60.29) (Figure 3). As shown, this model had slightly different key contributors, one being that *Tympanum – Ambiguous* was a more important variable than *DF Brokenness – Continuous*. These variables (and their accompanying factors) had larger sums of the absolute coefficients across all outcome levels than other variables although other variables were contributors as well.

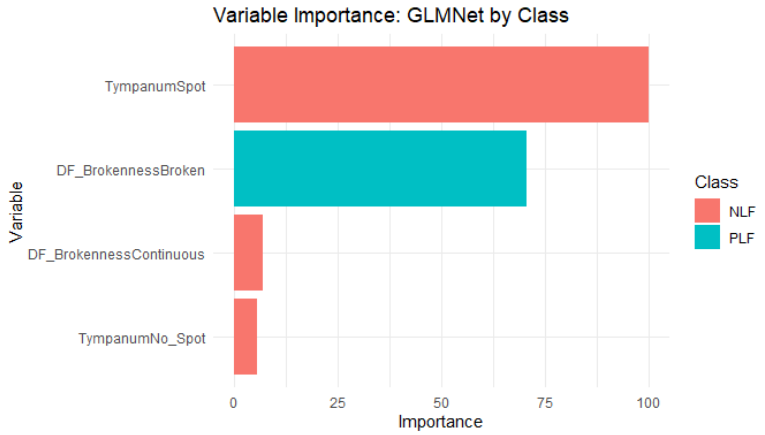


Figure 4: Variable importance for GLMNET model

The best tuned model for GLMNET with an overall accuracy of 0.914 (0.8859, 0.9371) had *alpha* = 1 and *Lambda* = 0.0886, which means this model used Lasso regularization and had moderate strength in our regularization penalty. GLMNET separates variable importance by species. The most important variable in classifying NLF is *Tympanum – Spot* (100) as well as *DF Brokenness Continuous* (7) and *Tympanum No Spot* (6), with substantially smaller importance. The most important variable for classifying PLF is *DF Brokenness – Broken* (71) (Figure 4). GLMNET did not designate any variable importance toward classifying HYB as no variables were considerably important in distinguishing HYB from other classes. All other predictors have zero scores for variable importance due to the Lasso penalty, which inherently carries out feature selection.

From Table 3, we see that all three models performed almost perfectly in their classification of NLF. We can see this visually through our confusion matrices where NLF had the highest counts of true positives for all three models and rarely was misclassified (Figure 2). This suggests that NLF can especially be categorized based on morphology alone. Our models across the board performed second best at classifying PLF and the worst for HYB. We reason that this may be a result of HYB having a large spread of traits across variables especially with PLF (as shown in Figure 1 and Table 1), which is expected with species hybridization. Moreover, we recommend for further classification initiatives to use our GLMNET model, which resulted in the highest accuracy metrics across species (especially with substantially higher metrics for HYB with markedly higher metrics than our other models as shown in Table 3). In addition to high accuracy metrics, our reduced model may prove more cost effective as well since researchers can prioritize virtually only two variables: *Tympanum* and *DF_Brokenness* without sacrificing prediction performance. These models, along with our exploratory analysis, support that if researchers were short on time and resources, they could use the following classification process. If a frog has no spot on its tympanum, then it's NLF. If the frog has a spot on its tympanum and its dorsal front was broken, then it's PLF. Otherwise, it's likely HYB. Hopefully, this analysis is helpful for quick, cost-effective classification, and we look forward to seeing if further data collection supports our claims.

Authors' Statements

Both authors contributed to statistical analysis and writing for this report. Both authors read and approved the final report. Connor led the initiative to use the *caret* package for our statistical analysis. He also edited and refined the wording of our report, so that our analysis was portrayed accurately. Matthew led writing the report, explaining our models and analysis and citing sources when needed to support our study design and results. He also edited and double-checked the code to make sure results reflected research goals.

References

Aston, Z., Zachary C., L., Mu, L., & Alexander J., S. (2023). *Dive into Deep Learning*. Cambridge University Press.

Babu, A. (2025, March 13). *Why Accuracy Isn't Enough: A Guide to AI/ML Model Evaluation*. Medium. *https://medium.com/@aditib259/why-accuracy-isnt-enough-a-guide-to-ai-ml-model-evaluation-f97c0c394375*

Bobbitt, Zach. (2021, October 6). *What is Balanced Accuracy? (Definition & Example)*. Statology. *https://www.statology.org/balanced-accuracy/*

Campbell, J. J. (2025, July 10). *Lasso and Ridge Regression with Caret in R*. RPubs ; RStudio. *https://rpubs.com/uky994/1328850*

Cho, H., & Su, R. (n.d.). R: Calculates Gini Importance or Mean Decrease Impurity (same... R-Project.org. Retrieved June 23, 2026, from *https://search.r-project.org/CRAN/refmans/forestRK/html/importance.forestRK.html*

Frost, J. (2024, October 13). Multinomial Logistic Regression: Overview & Example. Statistics by Jim. *https://statisticsbyjim.com/regression/multinomial-logistic-regression/*

GeeksforGeeks. (2024, February 22). *Random Forest Algorithm in Machine Learning*. GeeksforGeeks. *https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/*

Google. (2024). *Classification: Accuracy, recall, precision, and related metrics*. Google for Developers. *https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall*

Hastie, T., Qian, J., & Tay, K. (2026, April 28). An Introduction to `glmnet`. Glmnet.stanford.edu. *https://glmnet.stanford.edu/articles/glmnet.html*

Kuhn, M. (2008). "Building Predictive Models in R Using the caret Package." *Journal of Statistical Software*, **28**(5), 1–26. *doi:10.18637/jss.v028.i05*. *https://www.jstatsoft.org/index.php/jss/article/view/v028i05*.

Kuhn, M. (2016). *Classification and Regression Training Description Misc functions for training and plotting classification and regression models*. *https://cran.r-project.org/web/packages/caret/caret.pdf*

Kuhn, M., & Johnson, K. (2016). Applied predictive modeling. Springer.

Kuhn, M. (2019). 15 Variable Importance | The caret Package. In *topepo.github.io*. *https://topepo.github.io/caret/variable-importance.html*

Prabhakaran, S. (2026, June 22). *caret varImp() in R: Rank Predictor Importance Scores*. R-Statistics.co. *https://r-statistics.co/caret-varImp-in-R.html*

ranger function - RDocumentation. (n.d.). *Www.rdocumentation.org*. *https://www.rdocumentation.org/packages/ranger/versions/0.16.0/topics/ranger*

Thankachan, K. (2022, August 6). *What? When? How?: ExtraTrees Classifier | Towards Data Science*. Towards Data Science. *https://towardsdatascience.com/what-when-how-extratrees-classifier-c939f905851c/*

Ward, F., Bailey, L., Newmark, E. (2026, May 19). "Comparative Morphological Analysis of a Leopard Frog Complex in Colorado", STAA 556 class presentation.

Appendix

```
#Retain this code chunk!!!
library(knitr)
library(kableExtra)
library(ggplot2)
library(dplyr)
library(emmeans)
library(effects)
library(pbkrtest)
library(table1)
library(tidyverse)
library(caret)
library(ranger)
library(MLmetrics)
library(C50)
library(gbm)
library(table1)
library(kernlab)
library(ggthemes)
library(flextable)

knitr::opts_chunk$set(echo = FALSE)
knitr::opts_chunk$set(message = FALSE)
knitr::opts_chunk$set(warning = FALSE)
knitr::opts_chunk$set(fig.width = 7, fig.height = 4)
#### Read data ####
# frogs <- read_csv("/Users/matthewblong/Desktop/MAS/SUMMER/LF30.csv")
#Matthew path
frogs <- read_csv("LF30.csv") #Connor path
#### Clean Data ####
frogs_clean <- frogs |>
  select(-c(Individual, SVL)) |>
  mutate(across(everything(), as.factor)) |>
  mutate(across(where(is.factor), ~ factor(str_replace_all(levels(.x), " ",
"_")[.x])))

# names(frogs_clean)
# replace spaces iwth underscores
names(frogs_clean) <- gsub("\\s+", "_", names(frogs_clean))
#### Either remove but also fine with keeping it since we can discuss why we
didn't use a train test split
#### Train/test split ####
# ind <- sample(1:50, ceiling(nrow(frogs) * 0.7))
# frogs_train <- frogs_clean[ind,]
# frogs_test <- frogs_clean[-ind,]

#### Merge left and right as constant result or ambiguous if mixed ####
frogs_clean_comb <- frogs_clean |>
  mutate(Tympanum = if_else(Left_Tympanum == Right_Tympanum, Left_Tympanum,
```

```
"Ambiguous"),
         DF_Insetness = if_else(DFL_Insetness == DFR_Insetness,
DFL_Insetness, "Ambiguous"),
         DF_Brokenness = if_else(DFL_Brokenness == DFR_Brokenness,
DFL_Brokenness, "Ambiguous")) |>
select(-c(Left_Tympanum, Right_Tympanum, DFL_Insetness, DFL_Brokenness,
DFR_Insetness, DFR_Brokenness)) # variable reduction helps prevent
overfitting

#### Some models need encoded variables
dummy <- dummyVars(ID ~ ., data = frogs_clean, sep = "_")
dummy_comb <- dummyVars(ID ~ ., data = frogs_clean_comb, sep = "_")
frogs_encoded <- predict(dummy, frogs_clean) |>
  as.data.frame()
frogs_encoded_comb <- predict(dummy_comb, frogs_clean_comb) |>
  as.data.frame()
frogs_encoded$ID <- frogs_clean$ID
frogs_encoded_comb$ID <- frogs_clean_comb$ID
# ggplot(frogs_clean, aes(x = ID, fill = ID)) +
#   geom_bar() +
#   scale_fill_manual(values = colorblind_pal()(8)[c(2,3,4)])
# ggplot(frogs_clean, aes(x = `Left Tympanum`, fill = ID)) +
#   geom_bar()

plt <-
  frogs_clean_comb |>
  mutate(Thigh_Pattern = if_else(str_detect(tolower(Thigh_Pattern),
'individually'), 'Individual', Thigh_Pattern)) |>
  mutate(Upper_Lip_Stripe = if_else(str_detect(tolower(Upper_Lip_Stripe),
'distinct'), str_extract(Upper_Lip_Stripe, '[^_]+'), Upper_Lip_Stripe)) |>
  pivot_longer(-ID, names_to = "predictor", values_to = "value") |>
  ggplot(aes(x = value, fill = ID)) +
  geom_bar(position = "fill") +
  facet_wrap(~ predictor, scales = "free_x") +
  labs(y = "Proportion", title = "Predictor vs Species") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_manual(values = colorblind_pal()(8)[c(2,3,4)])
  # scale_fill_colorblind()

plt
tab <- table1(~.|ID, data=frogs_clean_comb)
tab
# knitr::kable(tab, caption = "Table 1")
### Variable Description Table ###
variables <- data.frame(
  Variable = c(
    "ID", "SVL", "DF Insetness", "DF Brokenness", "Nose Spot", "Thigh
Pattern", "Tympanum",
    "Upper Lip Stripe"),
```

```
  Description = c(
    "The species of each frog",
    "Snout to vent length of each frog ",
    "Categorizes the Dorsal Front Insetness ",
    "Categorizes the Dorsal Front Brokenness ",
    "Indicates whether the frog has a spot on its nose",
    "Describes the thigh pattern on each frog",
    "Describes whether the tympanum of each frog has a spot",
    "Categorizes the distinctness of a stripe on each frog's upper lip"),

  Class = c(
    "Factor (3 levels: NLF, PLF, HYB)", "Numeric", "Factor (3 levels: Inset,
Straight, Ambiguous)", "Factor (3 levels: Broken, Continuous, Ambiguous)",
"Factor (2 levels: 1, 0)",
    "Factor (3 levels: Reticulated, Individually Spotted, Ambiguous)",
"Factor (3 levels: Spot, No Spot, Ambiguous)", "Factor (3 levels: Distinct in
front of eye, Indistinct in front of eye, Ambiguous)"),

  Role = c(
    "Response",
    "Not Applicable",
    "Predictor",
    "Predictor",
    "Predictor",
    "Predictor",
    "Predictor",
    "Predictor"
  )
)

kable(
  variables,
  # caption = "Table 2: Description of all variables measured with units and
role in data analysis",
  booktabs = TRUE,
  longtable = FALSE
) #%>%
  # kable_styling(
  #   latex_options = c("hold_position"),
  #   font_size = 9
  # )|>
  # row_spec(0, bold = TRUE)|>
  # column_spec(1, width = "2.5cm")|>
  # column_spec(2, width = "6cm")|>
  # column_spec(3, width = "2.5cm")|>
  # column_spec(4, width = "3cm")
#### RF Model ####
set.seed(1)
```

```r
#set train control
tc <- trainControl(
  method = "repeatedcv", #repeated cross validation. Splitting data into
folds and then training to test performance
  number = 5, # splitting data into 5 folds. 1 fold will be tested
  repeats = 10, # The entire k-fold CV is repeated 10 times
  classProbs = TRUE,
  summaryFunction = multiClassSummary, # returns several metrics; more than
just accuracy
  savePredictions = "final" # saves best model
)

#set tune grid
tg <- expand.grid(
  mtry = 2:8, # number of randomly selected predictors at each tree split
  splitrule = c("gini", "extratrees"), # how to compare splits. extratrees
uses more randomness
  min.node.size = 1 # minimum number of obs for a split in the tree. # 1:5?
)

#run model
frogs_rf <- train(
  ID ~ .,
  data = frogs_clean_comb,
  method = "ranger",
  metric = "Mean_Balanced_Accuracy", #Calculates the average accuracy for
each class or species. Smaller class sizes aren't less valuable like in
overall accuracy
  trControl = tc,
  tuneGrid = tg,
  num.trees = 500, #default value and no need to change
  importance = "impurity" #measures how each split reduces class mixing
)

#check best tunes
# frogs_rf$bestTune

#check variable importance
# varImp(frogs_rf) # scaled raw values. Directionless. it is relative so a
score of 50 is half as important as 100
# plot(varImp(frogs_rf))

#build confusion matrices
cm_rf <- confusionMatrix(frogs_rf$pred$pred, frogs_rf$pred$obs, mode =
"everything")
# cm_rf

cm_rf_plt <- ggplot(as.data.frame(cm_rf$table), aes(x = Reference, y =
Prediction, fill = Freq)) +
```

```r
  geom_tile() +
  geom_text(aes(label = Freq), color = "black", size = 5, fontface = "bold")
+
  scale_fill_gradient(low = "white", high = "#2166AC") +
  labs(title = "Confusion Matrix", x = "Actual", y = "Predicted") +
  theme_minimal()

cm_rf_plt
#### Multinomial Logisitic Regression ####
set.seed(1)

# set tune grid
tg_mlr <- expand.grid(decay = c(0, 0.001, 0.01, 0.1, 1)) #L2 weight. 0 is
unpenalized and the larger values add more regularization

# run model
frogs_mlr <- train(
  ID ~ .,
  data = frogs_encoded_comb,
  method = "multinom",
  metric = "Mean_Balanced_Accuracy",
  trControl = tc,
  tuneGrid = tg_mlr,
  trace = FALSE
)
# names(frogs_mlr$trainingData) <- gsub("/", "",
names(frogs_mlr$trainingData))

# check best tune
# frogs_mlr$bestTune

# variable interpretation
temp <- summary(frogs_mlr)
temp$coefnames <- str_extract(temp$coefnames, "[A-Za-z\\s]+")
# temp$coefficients

# variable importance
# varImp(frogs_mlr)

# confusion matrix
cm_mlr <- confusionMatrix(frogs_mlr$pred$pred, frogs_mlr$pred$obs, mode =
"everything")

cm_mlr_plt <- ggplot(as.data.frame(cm_mlr$table), aes(x = Reference, y =
Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), color = "black", size = 5, fontface = "bold")
+
  scale_fill_gradient(low = "white", high = "#2166AC") +
```

```r
  labs(title = "Confusion Matrix", x = "Actual", y = "Predicted") +
  theme_minimal()

cm_mlr_plt
#### GLMNET ####
set.seed(1)

#set tune grid
tg_glmnet <- expand.grid(
  alpha = seq(0, 1, by = 0.25),  # mix of Ridge (0) and LASSO (1)
  lambda = 10 ^ seq(-4, 0, length = 20) #regularization strenngth. Larger
values shrink coefficients more
)

# run model
frogs_glmnet <- train(
  ID ~ .,
  data = frogs_encoded_comb,
  method = "glmnet", #### try others
  metric = "Mean_Balanced_Accuracy",
  trControl = tc,
  tuneGrid = tg_glmnet,
  trace = FALSE
)

# check best tune
# frogs_glmnet$bestTune

# Extract coefficients at the best lambda
# coef(frogs_glmnet$finalModel, s = frogs_glmnet$bestTune$lambda)

# variable importance
# varImp(frogs_glmnet)

# confusion matrix
cm_glmnet <- confusionMatrix(frogs_glmnet$pred$pred, frogs_glmnet$pred$obs,
mode = "everything")
# cm_glmnet

cm_glmnet_plt <- ggplot(as.data.frame(cm_glmnet$table), aes(x = Reference, y
= Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), color = "black", size = 5, fontface = "bold")
+
  scale_fill_gradient(low = "white", high = "#2166AC") +
  labs(title = "Confusion Matrix", x = "Actual", y = "Predicted") +
  theme_minimal()

cm_glmnet_plt
```

```r
extract_byclass <- function(cm, model_name) {
  cm$byClass |>
    as.data.frame() |>
    rownames_to_column("Species") |>
    mutate(
      Species = gsub("Class: ", "", Species),
      Model = model_name
    ) |>
    select(Model, Species, Sensitivity, Specificity, Precision, F1, `Balanced
Accuracy`)
}

byclass_all <- bind_rows(
  extract_byclass(cm_rf, "Random Forest"),
  extract_byclass(cm_mlr, "MLR"),
  extract_byclass(cm_glmnet, "GLMNET")
) |>
  pivot_longer(cols = c(Sensitivity, Specificity, Precision, F1, `Balanced
Accuracy`),
               names_to = "Metric",
               values_to = "Value") |>
  pivot_wider(names_from = Model, values_from = Value) |>
  arrange(Species, Metric)

overall_rows <- byclass_all |>
  group_by(Metric) |>
  summarise(
    `Random Forest` = mean(`Random Forest`, na.rm = TRUE),
    `MLR` = mean(`MLR`, na.rm = TRUE),
    `GLMNET` = mean(`GLMNET`, na.rm = TRUE)
  ) |>
  mutate(Species = "Overall") |>
  select(Species, Metric, everything())

final_table <- bind_rows(byclass_all, overall_rows)

flextable(final_table) |>
  merge_v(j = "Species") |>
  valign(j = "Species", valign = "top") |>
  set_caption("Table 4: Accuracy Statistics by Species and Model") |>
  colformat_double(digits = 3) |>
  theme_vanilla() |>
  autofit()

# knitr::kable(byclass_all,
#              caption = "Table 4: Accuracy Statistics by Species and Model",
#              digits = 3,
#              row.names = FALSE)
```

```r
vi_rf <- varImp(frogs_rf)$importance |>
  as.data.frame() |>
  rownames_to_column("Variable") |>
  mutate(Model = "Random Forest")

vi_mlr <- varImp(frogs_mlr)$importance |>
  as.data.frame() |>
  rownames_to_column("Variable") |>
  mutate(Model = "MLR")

vi_both <- bind_rows(vi_rf, vi_mlr)

ggplot(vi_both, aes(x = reorder(Variable, Overall), y = Overall, fill =
Model)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  facet_wrap(~ Model, scales = "free_x") +
  labs(title = "Variable Importance: Random Forest and MLR", x = "Variable",
y = "Importance") +
  theme_minimal() +
  theme(legend.position = "none")

vi_glmnet_long <- varImp(frogs_glmnet)$importance |>
  as.data.frame() |>
  rownames_to_column("Variable") |>
  pivot_longer(-Variable, names_to = "Class", values_to = "Importance") |>
  filter(Importance > 0)

ggplot(vi_glmnet_long, aes(x = reorder(Variable, Importance), y = Importance,
fill = Class)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(title = "Variable Importance: GLMNet by Class", x = "Variable", y =
"Importance") +
  theme_minimal()
```